

Generating a sparse global codebook

Support evidence and coordinate-wise amplitude generation

Stage	5
Updated	18 September 2026
Status	Support tests complete. Coordinate-amplitude job 031 locally checked; HPC results pending.

1 What we are trying to preserve

A B -bit message chooses one of $M = 2^B$ columns of a common codebook:

$$\Phi = [\phi(w_0), \dots, \phi(w_{M-1})] \in \mathbb{R}^{n \times M}, \quad y = \Phi a + z = \sum_{k=1}^K \phi(w_k) + z, \quad \|\phi(w)\|_2^2 = 1.$$

Here $a \in \mathbb{Z}_+^M$ counts transmitted messages and sums to K . Repeated messages add multiplicity. Bits label messages, not channel amplitudes: the transmitted coordinates are real numbers in these experiments.

The useful empirical result is that a sparse global matrix can perform close to a dense matrix, whereas a particular ODMA support restriction performs substantially worse at the same density. Job 027 at $B = 12, n = 256$ gave D0 mean high-SNR PUPE 0.2935 for dense, 0.2932 for arbitrary support size 64, and 0.5363 for four-mask ODMA with the same support size. We seek inexpensive structure that preserves useful codebooks, not sparsity alone.

Storing even just T nonzeros per message costs $T2^B$ values. It cannot reach $B = 100$. We therefore replace stored columns by a rule $w \mapsto \phi(w)$ and test each restriction against the explicit parent at small B . D0/D1 provide a common receiver for these comparisons; neither is certified optimal. A compact forward rule does not supply a fast inverse for a noisy sum of many messages. That separate problem remains open.

2 Support: one selected resource in each table

Partition $n = TR$ coordinates into T tables of $R = 2^r$ coordinates. For every message,

$$h_t(w) = A_t w \oplus b_t, \quad A_t \in \mathbb{F}_2^{r \times B}, \quad b_t \in \mathbb{F}_2^r, \quad \rho_t(w) = tR + \text{integer}(h_t(w)).$$

$\mathbb{F}_2$ means binary arithmetic, with XOR as addition. Tables are indexed $t = 0, \dots, T-1$. Every message uses one position per table: support size T , density $1/R$. These tables are not independently chosen payload sections; each hash can depend on all B bits.

One-per-table masks restrict all size- T masks, and affine hashes further restrict message-to-table assignments. Neither concession is proved free. Independent uniform table choices have expected pair overlap $T/R = T^2/n$, matching independent uniform size- T supports. Equal means do not guarantee equal tails or performance.

We impose two exact binary rank conditions:

$$\text{rank}_{\mathbb{F}_2}(A_t) = r, \quad \text{rank}_{\mathbb{F}_2} \begin{bmatrix} A_0 \\ \vdots \\ A_{T-1} \end{bmatrix} = B.$$

The first gives equal bin populations across all messages. The second ensures distinct messages have distinct full support tuples, requiring $Tr \geq B$. For $d = w \oplus v \neq 0$, overlap in table t occurs exactly when $A_t d = 0$. At small B , enumerating differences permits selection by overlap tails. Job 031 retains the previous best-of-128 hash search. This exhaustive selection is not a large- B algorithm; large- B forward checks instead sample rank-valid banks without enumeration.

Evidence status: Jobs 028 and 029 returned their 36 and 20 expected runs. Under joint training at $B = 14, n = 256$, selected hash minus arbitrary sparse PUPE was approximately $-0.0003/-0.0103$ for D0/D1 at $T = 16$, and $+0.0026/+0.0025$ at $T = 32$. The support restriction is a reasonable working choice at these tested points, not a universal theorem.

3 Amplitude: each coordinate has its own lookup

Keep the support fixed. Give table t a binary projection and a scalar lookup bank:

$$P_t \in \mathbb{F}_2^{J \times B}, \quad V \in \mathbb{R}^{T \times 2^J}, \quad u_t(w) = \text{integer}(P_t w), \quad g_t(w) = V_{t, u_t(w)}.$$

One message chooses T scalar entries, generally from different bank columns. Gather and normalize:

$$g(w) \in \mathbb{R}^T, \quad \alpha(w) = \frac{g(w)}{\|g(w)\|_2}, \quad \boxed{\phi(w) = \sum_{t=0}^{T-1} \alpha_t(w) e_{\rho_t(w)}}.$$

Positions are distinct, so $\|\phi(w)\|_2^2 = \sum_t \alpha_t(w)^2 = 1$ for every message. For the exceptional $g = 0$ case the implementation defines $\alpha = e_0$, also unit energy. This is not expected with continuous initialization; if reached, actual nonzero support decreases.

For $B = 14, n = 256, T = 32, R = 8, J = 4$, the support maps have shape $(32, 3, 14)$, amplitude maps $(32, 4, 14)$, and V has shape $(32, 16)$: 512 real amplitude parameters rather than $32 \cdot 2^{14} = 524,288$. Storage is $O((T(r+J)B + T2^J))$; generating one sparse column costs $O((T(r+J)B + T))$ without visiting other messages.

3.1 The equivalent explicit matrix

For explanation only, enumerate all messages. Define one-hot lookup matrices

$$(S_t)_{jm} = \mathbf{1}\{j = h_t(w_m)\}, \quad S_t \in \{0, 1\}^{R \times M}, \quad (Q_t)_{um} = \mathbf{1}\{u = u_t(w_m)\}, \quad Q_t \in \{0, 1\}^{2^J \times M}.$$

The raw amplitude matrix $G \in \mathbb{R}^{T \times M}$ has row $G_{t,:} = V_{t,:} Q_t$. For nonzero columns let $c_m = 1/\|G_{:,m}\|_2$. Then

$$\boxed{\Phi = \begin{bmatrix} S_0 \text{diag}(G_{0,:}) \\ \vdots \\ S_{T-1} \text{diag}(G_{T-1,:}) \end{bmatrix} \text{diag}(c) \in \mathbb{R}^{TR \times M}}.$$

S_t, Q_t, G, Φ are conceptual objects, not required generator storage. The small- B adapter caches row/label indices to execute this matrix and its adjoint exactly. Current D0/D1 still score M messages.

3.2 Learning and remaining restrictions

A_t, b_t, P_t are fixed discrete choices. Fixed-amplitude runs train only the decoder; joint runs train V too. For $g \neq 0$, normalization has Jacobian

$$\frac{\partial \alpha}{\partial g} = \frac{I_T - \alpha \alpha^\top}{\|g\|_2}.$$

The lookup routes gradients into $V_{t, u_t(w)}$. Messages reusing an entry accumulate gradients there; XOR needs no derivative. A stored bank column is *not* generally a codeword's amplitude vector, so it must not be projected to unit norm after updates.

Scalar reuse is still a restriction: messages sharing $P_t w$ share one raw amplitude, but their other coordinates can differ. Each P_t is a prefix of a fixed invertible $B \times B$ basis. Repeating bank entries embeds a J -bit model in the $(J+1)$ -bit model. At $J = B$, each table's label is bijective and arbitrary amplitudes on the chosen supports are represented exactly. Model-class equality does not guarantee optimization reaches its best member. At fixed J , changing the label maps gives a different restricted family, not a guaranteed superset of the previous one.

4 Algebraic safeguards and initialization

The projections satisfy the following ranks over $\mathbb{F}_2$:

$$\text{rank}(P_t) = J, \quad \text{rank}(P_{\text{stack}}) = \min(TJ, B), \quad \text{rank} \begin{bmatrix} A_t \\ P_t \end{bmatrix} = \min(B, r + J).$$

These balance each bank’s label use, remove avoidable whole-label aliases, and preserve as many amplitude labels as possible within each hash bin. When $r + J \leq B$, bin and amplitude labels are independent for uniform messages. When $r + J > B$, full independence is impossible.

For $d = w \oplus v$, sharing both a resource and its raw amplitude entry requires $A_t d = P_t d = 0$. The joint kernel has dimension $B - \text{rank}[A_t; P_t]$, so maximal rank minimizes this alias set. It does not minimize every codeword correlation: different bank entries can have similar values and normalization couples all tables.

Coordinate labels also avoid a shared-factor rank bound. With one common lookup Q , the raw matrix $G = VQ$ has real rank at most $\min(T, 2^J)$. Different Q_t need not obey that bound: $T = 32, J = 4$ can allow amplitude rank 32 rather than at most 16. Nonzero-column normalization preserves this rank. The old *physical* matrix Φ need not have rank at most 16, however: table-specific support masking changes that argument.

4.1 Why center and balance a small bank?

Only 2^J Gaussian draws populate each table. At $J = 4$, its sample mean has standard deviation $1/4$; its sample energy also fluctuates. Reusing that short bank does not average away the table-specific bias. Initialize by

$$\mu_t = 2^{-J} \sum_u V_{tu}, \quad s_t^2 = 2^{-J} \sum_u (V_{tu} - \mu_t)^2, \quad V_{tu} \leftarrow (V_{tu} - \mu_t)/s_t.$$

Each raw bank now has mean zero and mean-square one. Full-rank labels make these exact averages over uniform messages too. Individual magnitudes still vary. Subsequent codeword normalization need not preserve exact table-energy equality, so row energy remains a measured diagnostic.

This is an *initialization*, not an ongoing constraint: learned banks can change means and energy allocation. The motivation is finite-bank bias and observed geometry, not a claim that all optimal encoders are balanced. A preliminary two-seed frozen-D0 probe gave PUPE about 0.4322 for balanced coordinate $J = 4$ versus 0.4306 for unrestricted amplitudes on its restricted four-cell grid. It motivates matched training, not a non-inferiority claim.

Useful physical geometry is broader than binary ranks: balanced row energy, small harmful correlation tails, well-conditioned active-column Gram matrices, and separation between different K -message sums. Since $M \gg n$, Φ cannot have full column rank. Full row rank and isolated-message injectivity alone cannot certify multiuser recovery.

5 Job 031: test amplitudes before changing the receiver

Keep $B = 14$, $n = 256$, $T = 32$, selected supports, global D0/D1, and paired message/noise streams fixed. Three seeds and two decoders give 72 rows:

Cases per seed and decoder	Question
Coordinate $J = 4, 8, 10$, balanced; fixed and joint	How much sharing preserves the unrestricted reference?
Coordinate $J = 4$, raw; fixed and joint	Does initialization explain a low- J penalty?
Shared-label $J = 4$, raw and balanced; fixed	Are separate labels helpful beyond balancing alone?
Unrestricted $J = 14$, raw; fixed and joint	Matched-budget parent with the same normalization/optimization.

Job 030 remains available as historical evidence, not the proposed route. All new rows use at most 120 epochs, patience five on deterministic validation loss, and best-checkpoint restoration including epoch zero. Spectral calibration uses a fixed starting vector. Fixed and joint runs have equal maximum data budgets but may stop at different epochs.

Compare each family with its same-seed, same-decoder, same-training-mode parent. Report all load/SNR cells, the 8/12-dB aggregate, paired seed gaps, stopping epochs, and pre/post geometry. Three seeds are useful replication, not a guarantee that small gaps are absent. Do not average away high-load failures.

Evidence status: Local checks cover endpoint equality, nesting, binary ranks, forward/adjoint equality, gradients, checkpoint round trips, exact energy, and sampled $B = 100$ generation. Twelve smoke paths and fixed/joint D0/D1 mini learning runs pass. These validate execution and early learning only; HPC results are pending.

The next decision is whether coordinate-wise scalar reuse retains the useful explicit-codebook performance. Only afterwards should a scalable inverse be added and its separate recovery loss measured. No sign-only, mutual-information, or new decoder construction is part of this batch.